

COMPLETE SYSTEM WORKFLOW

From Google Form Submission to Frontend Dashboard

Every step, every component, every decision — explained

Feedback Intelligence Dashboard

Futureense Institute · Full Stack Architecture · March 2026

System Architecture Overview

The Feedback Intelligence Dashboard is a four-layer system. Data enters through a Google Form, travels through a Google Sheets intermediary, gets processed by a FastAPI backend connected to PostgreSQL, and is finally displayed on a React frontend. Each layer has a distinct responsibility and communicates with the next through well-defined interfaces.

Layer	Technology	Role	Port / Location
Input	Google Forms	Student feedback collection	Google Cloud
Buffer	Google Sheets	Intermediary data store, synced live	Google Cloud
Backend	FastAPI + Python	Business logic, MPI engine, AI integration	localhost:8000
Database	PostgreSQL	Persistent storage of all entities and feedback	localhost:5432
Frontend	React + TypeScript	Interactive dashboard and visualisations	localhost:5173

End-to-end data journey — summary:

1	Student fills Google Form	Rating 1-5, structured response, free-text comment
2	Google Sheets receives it	Automatically added to 'Form Responses 1' worksheet
3	APScheduler fires every 60s	sync_feedback() function polls Sheets for new rows
4	Sync validates & resolves entities	Program → Cohort → Mentor → Session → Student lookup chain
5	Feedback inserted into PostgreSQL	Batched commit, duplicate prevention via timestamp check
6	FastAPI serves MPI calculations	kpi_service.py computes Q, C, R, T, MPI, Z-score, category
7	AI generates executive brief	Gemini 2.5 Flash reads metrics + student comments
8	React fetches and renders	Silent 30s auto-refresh keeps dashboard live

1

Student Input — Google Form

Layer: Input | Technology: Google Forms | Location: Google Cloud

Everything begins with a student completing a Google Form after attending a session. The form is the only data entry point — no data is manually entered anywhere in the system.

What the student fills in:

Field	Type	Values	Maps to DB column
Timestamp	Auto	Exact datetime of submission	feedback.submitted_at
Email Address	Text	Student email address	students.email
Student Name	Text	Full name	students.name
Program	Dropdown	AI & ML / Data Engineering / MBA Analytics	programs.name (lookup)
Cohort	Dropdown	e.g. AI & ML Cohort 1	cohorts.name (lookup)
Meeting / Week	Number	1 through 10	sessions.week_number (lookup)
Instructor	Dropdown	Mentor name e.g. Rohan Das	mentors.name (lookup)
Session Rating	Scale 1-5	1 = Poor, 5 = Excellent	feedback.rating_numeric
How was the session?	Dropdown	Excellent / Good / Average / Poor	feedback.structured_response
Additional feedback	Text (opt)	Free text comment	feedback.comment

■ Key Design Decision

The form uses dropdown fields (not free text) for Program, Cohort, Instructor, and Rating. This ensures every submission can be matched to existing database records exactly. Free text would introduce spelling variations that would break the entity lookup chain.

Response lands in Google Sheets automatically



2

Google Sheets Buffer

Layer: Intermediary | Technology: Google Sheets | Spreadsheet: 'Futureense Student Feedback Responses'

Google Forms automatically appends each submission as a new row in a Google Sheets spreadsheet. This spreadsheet acts as the live queue — a buffer between the public-facing form and the private database. The backend reads from this sheet rather than querying Google Forms directly, because the Sheets API is simpler, faster, and supports reading all rows in a single call.

What the sheet looks like (each row = one form submission):

Column	Example Value	Notes
Timestamp	2026-03-15 14:32:07	Auto-generated by Google Forms on submit
Email Address	student42@mail.com	Used to find/create Student record
Student Name	Student 42	Stored in students.name
Program	AI & ML	Matched to programs table
Cohort	AI & ML Cohort 1	Matched to cohorts table scoped to program
Meeting / Week	7	Integer — matched to sessions.week_number
Instructor	Rohan Das	Matched to mentors.name
Session Rating	4	Cast to float for rating_numeric
How was the session?	Good	Stored as structured_response verbatim
Any additional feedback	Very clear explanation	Stored as comment, can be empty string

■■ Column Name Whitespace Issue (Bug Fixed)

Google Forms sometimes pads column names with leading/trailing spaces.

A raw dict lookup like `row['Timestamp']` would fail with a `KeyError`.

Fix: `normalize_keys()` strips all whitespace from column names before any lookup.

Code: `row = {key.strip(): value for key, value in raw_row.items()}`

APScheduler polls this sheet every 60 seconds



3

Backend Sync Service

Layer: Backend | File: sync_service.py | Trigger: APScheduler every 60s

The sync_service.py module is the heart of the data pipeline. It runs automatically in the background every 60 seconds, reads every row from Google Sheets, and decides what to insert into PostgreSQL. It must handle duplicates, missing entities, malformed data, and authentication — all without crashing the scheduler.

Step-by-step execution of sync_feedback():

3.1 Authentication & Sheet Access

Uses Google Service Account credentials from credentials.json
Scopes: spreadsheets (read) + drive (required for file listing)
gspread.authorize() returns a client; client.open(name).worksheet(name) returns the sheet
sheet.get_all_records() fetches all rows as a list of dicts in one API call

3.2 Parse Timestamp

```
submitted_time = parser.parse(row['Timestamp'])
```

dateutil.parser handles any timestamp format Google Sheets might produce

- This value becomes the primary duplicate-check key

IMPORTANT: This must happen BEFORE the duplicate check query

3.3 Duplicate Check

```
Query: db.query(Feedback).filter(Feedback.submitted_at == submitted_time).first()
```

- If a Feedback row already exists with this exact timestamp, skip the row with 'continue'
- This prevents any row from being inserted twice across sync cycles

The unique constraint (student_id, session_id, submitted_at) provides DB-level backup protection

3.4 Extract & Validate Fields

```
program_name = row['Program'].strip()
cohort_name = row['Cohort'].strip()
week_number = int(row['Meeting / Week'])
```

- If program_name is empty string, log warning and skip row — do not crash

3.5 Entity Resolution Chain

```
Program lookup: db.query(Program).filter(Program.name == program_name).first()
Cohort lookup: db.query(Cohort).filter(Cohort.name == cohort_name, Cohort.program_id ==
program_id).first()
Mentor lookup: db.query(Mentor).filter(Mentor.name == row['Instructor'].strip()).first()
Session lookup: db.query(Session).filter(week_number, cohort_id, mentor_id).first()
```

- If ANY lookup returns None: log warning, skip row — entity must pre-exist in DB

3.6 Student Get-or-Create

- Query students by email: `db.query(Student).filter(Student.email == row['Email Address']).first()`
 If student does not exist: `create Student(name, email, cohort_id)` and `db.flush()` (not commit)
`db.flush()` assigns the `student.id` without committing – needed for the foreign key on Feedback
- Students are the ONLY entity auto-created; all others must pre-exist

3.7 Create Feedback Row

```
feedback = Feedback(
    rating_numeric = float(row['Session Rating']),
    structured_response= row['How was the session?'],
    comment = row['Any additional feedback'],
    submitted_at = submitted_time,
    collected_at = submitted_time,
    student_id = student.id,
    session_id = session.id
)
db.add(feedback) – queued but not yet committed
```

3.8 Batch Commit

After processing all rows in the loop: `db.commit()`

- ALL new feedback rows are committed in one transaction — not one commit per row
- This is ~100x faster for large batches and prevents partial writes
- If commit fails, entire batch rolls back — no partial data

`db.close()` releases the connection back to the pool

`logger.info` reports how many rows were inserted this cycle

■ What gets pre-loaded vs auto-created

Pre-loaded (must exist): Institution, Programs, Cohorts, Mentors, Sessions

Auto-created at sync time: Students (new students enrol continuously)

Auto-created at sync time: Feedback rows (the core fact being recorded)

If a row references a Program/Cohort/Mentor/Session not in DB: row is skipped with a warning log

Feedback is now in PostgreSQL — backend can query it



4

PostgreSQL Database Schema

Layer: Persistence | Technology: PostgreSQL + SQLAlchemy ORM | File: models.py

The database is the single source of truth. All seven tables are normalised with proper foreign keys and referential integrity. SQLAlchemy ORM maps Python classes to tables, allowing the backend to query using Python objects rather than raw SQL — with one exception: the flat export query in `reporting_service.py` uses raw SQL for performance.

Table	Key Columns	Relationships	Purpose
institutions	id, name	→ programs	Top-level entity. One institution owns all programs.
programs	id, name, institution_id	← institutions → cohorts	AI & ML, Data Engineering, MBA Analytics
cohorts	id, name, year, program_id	← programs → students, sessions	e.g. AI & ML Cohort 1. Two per program.
mentors	id, name	→ sessions	8 mentors. No cohort affiliation — can teach any.
students	id, name, email (unique), cohort_id	← cohorts → feedback	300 students. Email is the dedup key on sync.
sessions	id, topic, week_number, session_date, cohort_id, mentor_id	← cohorts, mentors → feedback	One session per week per cohort. 60 total.
feedback	id, rating_numeric, structured_response, comment, submitted_at, student_id, session_id	← students, sessions	Core fact table. 15,000 rows. Has unique constraint.

■ Unique Constraint on Feedback Table

The feedback table has: `UNIQUE(student_id, session_id, submitted_at)`

This means: one student cannot submit the same session rating twice at the same timestamp

This is the database-level backup to the `sync_service` timestamp duplicate check

If both checks somehow fail, the DB constraint prevents any corrupt data entering

SQLAlchemy relationships defined in `models.py`:

From	Attribute	To	Direction
Institution	programs	Program	One Institution → Many Programs
Program	cohorts	Cohort	One Program → Many Cohorts
Cohort	students	Student	One Cohort → Many Students
Cohort	sessions	Session	One Cohort → Many Sessions
Mentor	sessions	Session	One Mentor → Many Sessions

Session	feedback	Feedback	One Session → Many Feedback rows
Student	feedback	Feedback	One Student → Many Feedback rows

FastAPI queries the database on every API request



5

FastAPI Backend & MPI Engine

Layer: Backend | Files: api.py, kpi_service.py | Port: 8000

The FastAPI application is the brain of the system. It exposes REST endpoints that the React frontend calls to get data. The most important endpoint is GET /analytics, which triggers the MPI engine in kpi_service.py — a chain of database queries and mathematical calculations that produce the full performance profile for every mentor.

All API Endpoints:

Method	Endpoint	What it does	Called by
GET	/health	Returns {status: healthy} — used for connection checks	Frontend on load
POST	/sync	Manually triggers sync_feedback() — bypasses the 60s scheduler	Manual / testing
GET	/analytics	Full MPI breakdown for all mentors. Optional query params: ?program=, ?cohort=, ?mentor=	Every dashboard page
GET	/mentors/kpi	Raw KPI data for all mentors without filtering	Mentor Intelligence
GET	/mentors/trends	Weekly avg rating per mentor across all weeks	Trend charts, Leaderboard sparklines
GET	/insights	Rule-based text insights per mentor based on category	AI Insights page
GET	/mentor-feedback/{id}	Rating distribution, weekly avg, volume, attendance for one mentor	Mentor Detail page
GET	/mentor-intelligence/{id}	Triggers Gemini AI brief generation (cached after first call)	Mentor Detail page
GET	/mentor-comments/{id}	20 most recent student free-text comments for one mentor	Comment Explorer
GET	/reports/feedback	Flat JOIN query across all tables — used for CSV export	Export button

Inside kpi_service.py — MPI Calculation (calculate_mentor_kpis)

This function is the most computationally important piece of the backend. It runs once per GET /analytics call, pulling all required data from PostgreSQL in a series of optimised queries, then computing the four-dimensional MPI score for every mentor in sequence.

5.1 Institution-wide statistics query

```
db.query(func.avg(Feedback.rating_numeric), func.stddev_pop(Feedback.rating_numeric)).one()
```

- Returns: overall_avg and overall_std — used for Z-score calculation

If stddev is None (all ratings identical), default to 1 to avoid division by zero

5.2 Per-mentor aggregate query

```
db.query(Mentor.id, Mentor.name,
func.avg(Feedback.rating_numeric).label('avg_rating'),
func.count(Feedback.id).label('feedback_count'),
func.stddev_pop(Feedback.rating_numeric).label('rating_stddev'))
.join(Session).join(Feedback).group_by(Mentor.id).all()
```

One query returns avg, count, and stddev for ALL 8 mentors simultaneously

5.3 Quality Score (Q) — weight 45%

$$Q = (\text{avg_rating} - 1) / 4$$

Normalises the 1-5 scale to 0-1. Rating of 1 → Q=0. Rating of 5 → Q=1.

Clamp: $Q = \max(0, \min(Q, 1))$ — handles floating point edge cases

Example: avg_rating=4.6 → $Q = (4.6-1)/4 = 0.90$

5.4 Consistency Score (C) — weight 20%

$$C = 1 - (\text{stddev} / 2)$$

Penalises variance in ratings. High stddev = low consistency.

Dividing by 2 calibrates for the 1-5 scale range.

Clamp: $C = \max(0, \min(C, 1))$

Example: stddev=0.4 → C=0.80 (stable). stddev=1.8 → C=0.10 (volatile)

5.5 Reliability Score (R) — weight 20%

$$R = 1 - \text{math.exp}(-\text{feedback_count} / 1000)$$

- Exponential growth function — confidence grows fast at first, then levels off

At 500 feedbacks: R=0.39. At 1875: R=0.85. At 5000: R=0.99

- This component ensures low-data mentors don't score unrealistically high

5.6 Trend Score (T) — weight 15%

Fetch last 4 weeks of weekly avg ratings for this mentor (separate query per mentor)

```
weeks = [week_number, ...], ratings = [avg_rating, ...]
```

- Calculate linear regression slope manually:

```
x_mean = sum(weeks)/len(weeks), y_mean = sum(ratings)/len(ratings)
```

```
numerator = sum((weeks[i]-x_mean)*(ratings[i]-y_mean) for i in range(n))
denominator = sum((weeks[i]-x_mean)**2 for i in range(n))
slope = numerator/denominator (if denominator != 0, else 0)
T = clamp(slope * 6, -1, +1) – scale tiny slopes into meaningful range
If fewer than 2 weeks of data: slope=0, T=0 (neutral)
```

5.7 Composite MPI

```
MPI = 0.45*Q + 0.20*C + 0.20*R + 0.15*T
Confidence = C * R (how trustworthy is this MPI score?)
Z-score = (avg_rating - overall_avg) / overall_std
```

- All values rounded: MPI to 3dp, scores to 3dp, avg_rating to 2dp

5.8 Category Classification

```
if MPI >= 0.72 AND Z > 0: → 'Top Performer'
elif MPI < 0.55 OR Z < -1.5: → 'At Risk'
elif C < 0.5 AND avg > 3.5: → 'High Volatility'
elif MPI < 0.65: → 'Needs Attention'
else: → 'Stable'
```

- Order matters: At Risk and Top Performer are checked first as overrides

5.9 Sort and return

```
mentor_scores.sort(key=lambda x: x['mpi_score'], reverse=True)
```

- Returns a Python list of dicts — FastAPI automatically serialises to JSON
- Each dict contains: mentor_id, mentor_name, avg_rating, rating_stddev,
- feedback_count, quality_score, consistency_score, reliability_score,
- trend_score, mpi_score, confidence_score, z_score, category

JSON response sent to frontend over HTTP



6

AI Service — Gemini 2.5 Flash

Layer: Backend | File: ai_service.py | Trigger: GET /mentor-intelligence/{id}

The AI service runs when a user opens a Mentor Detail page. It fetches the mentor's full KPI data, pulls their 50 most recent student comments from the database, constructs a structured prompt, and sends it to Google Gemini 2.5 Flash. The response is a markdown-formatted executive brief that is cached in memory and returned to the frontend.

6.1 Cache Check

- `brief_cache` is a Python dict at module level: {}
- `if mentor_id in brief_cache: return cached brief immediately (no API call)`
- `First call for a given mentor_id is slow (~3-8 seconds); all subsequent calls are instant`
- `Cache persists for the lifetime of the backend process (cleared on server restart)`

6.2 Fetch MPI Data

- `mentor_data = calculate_mentor_kpis()` — runs the full MPI engine
- `Find this mentor: mentor = next((m for m in mentor_data if m['mentor_id'] == mentor_id), None)`
- Calculate rank: sorted by `mpi_score` descending, find index + 1
- Calculate `institutional_avg`: mean of all mentors' `avg_rating`

6.3 Fetch Student Comments

- `db.query(Feedback.comment, Feedback.rating_numeric, Feedback.submitted_at)`
- `.join(Session).filter(Session.mentor_id == mentor_id)`
- `.filter(Feedback.comment != '')` — exclude empty comments
- `.order_by(Feedback.submitted_at.desc()).limit(50).all()`
- Up to 50 most recent non-empty comments are passed to the AI
- This gives Gemini real student voice, not just numbers

6.4 Prompt Construction

- Structured prompt includes:
- Quantitative: name, MPI, category, confidence, trend, quality, consistency, reliability
- Comparative: `avg_rating`, `institutional_avg`, rank (e.g. 2 out of 8)
- Qualitative: numbered list of actual student comments with rating
- Output instructions: 6 fixed sections (Snapshot, Strengths, Improvement, Voice, Momentum, Recommendation)
- Tone directive: analytical, executive, actionable — no generic filler

6.5 API Call to Gemini

```
client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))
response = client.models.generate_content(model='gemini-2.5-flash', contents=prompt)
brief_text = response.text
brief_cache[mentor_id] = brief_text — store in cache for all future requests
```

Error handling: returns fallback message if quota exceeded or API fails

6.6**Response Returned**

- Returns: {mentor_id: int, brief: str}

The 'brief' string is raw markdown (headers with ##, bold text, bullet lists)

- Frontend renders it using the react-markdown library with full formatting

The AI brief displays in the ExecutiveBrief.tsx component with a Lottie bot animation while loading

AI brief JSON returned to frontend



7

React Frontend — Dashboard Rendering

Layer: Frontend | Technology: React 19 + TypeScript | Port: 5173

The React frontend is the layer users interact with. It makes HTTP requests to the FastAPI backend, manages state locally and through a React Context, and renders all data as an interactive dashboard. It never talks to the database directly — all data comes through the FastAPI API.

Frontend request lifecycle — what happens on page load:

7.1 App Startup

- `main.tsx` renders
DashboardContext initialises: `toasts=[]`, `lastSyncTime=null`
`App.tsx` sets up all routes with `AnimatePresence` for exit animations
- `Layout.tsx` renders immediately: sidebar nav, sync indicator, K button

7.2 InstitutionPage (/) — Level 1 Load

`useEffect` fires on mount: calls `load()` function
Parallel fetch: `fetch('/analytics')` + `fetch('/mentors/trends')` via `Promise.all()`
While loading: `setLoading(true)` shows Lottie animation `Loader` component
On success: `setData(analytics.mentor_breakdown)`, `setTrendData(trends)`
`setLastSyncTime(new Date())` updates sidebar sync indicator
`setInterval(load_silent, 30000)` starts the 30-second auto-refresh cycle

7.3 Analytics Query with Filters

URL construction: `const params = new URLSearchParams()`
`if (program) params.append('program', program)`
`if (cohort) params.append('cohort', cohort)`
`fetch('/analytics?' + params)` – FastAPI filters the MPI results server-side
On cohort select: `setSearchParams` updates URL, `useEffect` dependency re-fires `load()`

7.4 Silent Refresh (every 30 seconds)

- `load(silent=true)` is called by `setInterval`
When `silent=true`: skip `setLoading(true)` – no spinner, no layout shift
- Data updates in place: charts re-render smoothly with new values
 - `pushToast('Data refreshed')` shows a 3-second notification
`setLastSyncTime(new Date())` resets the sidebar counter
 - `clearInterval` cleanup runs on component unmount to prevent memory leaks

7.5 Mentor Detail Page — /mentor/:mentorId

`useParams()` extracts `mentorId` from the URL
Three parallel fetches via `Promise.all()`:
`fetch('/analytics?mentor=' + mentorId)` – MPI scores

`fetch('/mentors/trends')` – weekly trend data for this mentor
`fetch('/mentor-feedback/' + mentorId)` – charts, distribution, attendance
 Separately: `fetch('/mentor-intelligence/' + mentorId)` for AI brief (`setAiLoading`)
 All data stored in separate `useState` hooks, components render independently

7.6 State Management Architecture

Global (`DashboardContext`): `toasts[], lastSyncTime, pushToast(), setLastSyncTime()`
 Page-level: `data, trendData, loading, error` – local `useState` per page
 URL state: program and cohort drill-down via `useSearchParams()`

- No Redux or external state library — React's built-in tools are sufficient

Error state: if `fetch` fails, `setError(true)` shows `ErrorState` component with `retry` button

7.7 Chart Rendering (Recharts)

- All charts are Recharts components inside `ResponsiveContainer` (100% width, fixed height)
- `TrendChart`: `LineChart` with one Line per mentor, data formatted as `[[week, mentorName: avg]]`
- `RiskQuadrant`: `ScatterChart` with custom dot rendering (glow circles, color by category)
- `MentorTrendChart`: `AreaChart` with gradient fill and `ReferenceLine` for institutional avg
- `SentimentPieChart`: `PieChart` (donut) with no stroke, no padding angle
- All charts have animated initial render and custom `Tooltip` with dark theme styling

7.8 KPI Card Animation (useCountUp hook)

Custom hook at `src/hooks/useCountUp.ts`

- Uses `requestAnimationFrame` for 60fps smooth counting
- Cubic ease-out easing: `progress = 1 - Math.pow(1 - t, 3)`
- Handles both integers (`toLocaleString`) and floats (`toFixed(2)`)
- Duration: 800ms. Re-animates on every value change (data refresh)
- `cancelAnimationFrame` cleanup on unmount prevents stale RAF callbacks

7.9 Command Palette (■K / Ctrl+K)

- `CommandPalette.tsx` renders as a fixed overlay (z-index 9999)
- Triggered by: keyboard shortcut listener in `Layout.tsx` `useEffect`
- On open: fetches `/mentors/kpi` to get mentor names list
 - Search: JavaScript `.includes()` filter on both page names and mentor names
- Navigation: `useNavigate()` from React Router – no page reload
- Keyboard: `ArrowUp/ArrowDown` move selection, `Enter` navigates, `Escape` closes

7.10 Export Features

- CSV: `exportUtils.ts` builds a string from mentor KPI array, creates `Blob`, triggers download
- PDF: `window.print()` — browser's native print dialog
- `index.css` `@media` print rules: hide sidebar, convert dark bg to white, add page breaks
- No third-party PDF library needed — the browser handles layout and printing

Complete End-to-End Timeline

This table traces one complete journey: a student submits feedback, and a manager sees the updated MPI score on the dashboard. Approximate timings for a normal operational cycle:

T+	What happens	System	File / Component
0s	Student submits Google Form	Google Forms	External
~1s	Response appended to Google Sheets as a new row	Google Sheets	External
0-60s	APScheduler waits for next 60s tick	Backend	api.py startup
~60s	sync_feedback() fires, reads all Sheets rows	Backend	sync_service.py
~61s	Timestamp duplicate check — row is new, proceed	Backend	sync_service.py:49
~61s	Entity resolution: Program→Cohort→Mentor→Session lookups	Database	sync_service.py:66-104
~61s	Student get-or-create by email, db.flush()	Database	sync_service.py:107
~62s	Feedback row created, db.add(feedback)	Database	sync_service.py:121
~62s	db.commit() — row is now permanently in PostgreSQL	Database	sync_service.py:134
0-30s	Frontend setInterval waits for next 30s tick	Frontend	InstitutionPage.tsx
~30s	load(silent=true) fires, fetch('/analytics') called	Frontend	InstitutionPage.tsx
~30s	FastAPI runs calculate_mentor_kpis()	Backend	kpi_service.py
~30s	MPI recalculated for all 8 mentors with new data included	Backend	kpi_service.py
~31s	JSON response returned to frontend	HTTP	FastAPI → axios
~31s	React re-renders: charts, scores, leaderboard all update	Frontend	Recharts + React
~31s	KPI counters animate to new values via useCountUp hook	Frontend	useCountUp.ts
~31s	Toast notification: 'Data refreshed' appears for 3 seconds	Frontend	DashboardContext.tsx
~32s	Sidebar sync indicator resets: 'just now'	Frontend	Layout.tsx
<1 min	Manager sees updated MPI, new feedback count on dashboard	Frontend	Leaderboard, KpiCard

Error Handling & Edge Cases

A production system must handle failures gracefully at every layer. Here is what happens when things go wrong, and how each failure is isolated so it does not cascade.

Failure Scenario	Where it occurs	What happens	User impact
Google Sheets API timeout	sync_service.py	Exception caught, logger.error(), scheduler continues — next tick tries again	At most 60s delay in new data
Form row has empty Program	sync_service.py:61	logger.warning(), row skipped with 'continue' — other rows still processed	One row lost, rest synced
Mentor name typo in form	sync_service.py:89	db.query(Mentor).first() returns None, logger.warning(), row skipped	One row lost, rest synced
Session not found in DB	sync_service.py:100	Row skipped with warning — session must be pre-seeded in DB	One row lost
PostgreSQL connection drops	database.py	pool_pre_ping=True reconnects automatically on next request	Temporary error response
Gemini API quota exceeded	ai_service.py	Returns fallback: 'AI quota exceeded. Please try again later.'	No AI brief, rest works
Backend unreachable	Frontend fetch()	catch block sets setError(true), shows ErrorState component with Retry button	Error page shown
No data for a section	Frontend	EmptyState component shown with emoji + message instead of empty chart	Friendly empty state
Duplicate form submission	sync_service.py:49	Timestamp check skips the row — unique DB constraint is backup	No duplicate in DB

Configuration & Environment

Environment variables (stored in .env file at project root):

Variable	Required	Example Value	Used in	Purpose
DATABASE_URL	Yes	postgresql://user:pass@localhost:5432/feedback_db	database.py	SQLAlchemy engine connection string
GOOGLE_API_KEY	Yes	Alza...	ai_service.py	Gemini 2.5 Flash API authentication
SYNC_INTERVAL	No	60 (default)	api.py	APScheduler interval in seconds

External files required:

- credentials.json — Google Service Account key file. Place at project root. Must have Sheets and Drive scopes enabled.
- Google Sheets spreadsheet — Named exactly 'Futureense Student Feedback Responses'. Must be shared with the service account email address.
- PostgreSQL — Must be running and accessible at the DATABASE_URL. Tables created by SQLAlchemy Base.metadata.create_all().
- Pre-seeded data — Institution, Programs, Cohorts, Mentors, and Sessions must exist before any sync can succeed. Run data_generator.py once to populate.

How to start both servers:

Backend:

```
uvicorn backend.main:app --reload --port 8000
```

Frontend:

```
cd dashboard-frontend && npm install && npm run dev
```

Both servers must be running simultaneously. The frontend (port 5173) talks to the backend (port 8000) via CORS-enabled HTTP requests.

This document covers the complete workflow from student form submission to frontend dashboard rendering. Every component, every function call, every decision point is traced in the order it executes in a live system.